

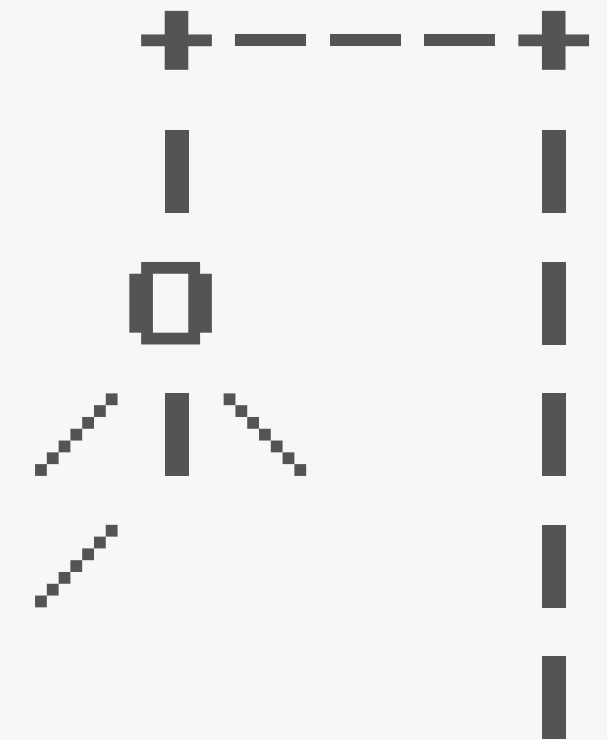
CATEGORIA: PROJETO DE REDES

HANGMAN

GAME

JOGADORES:

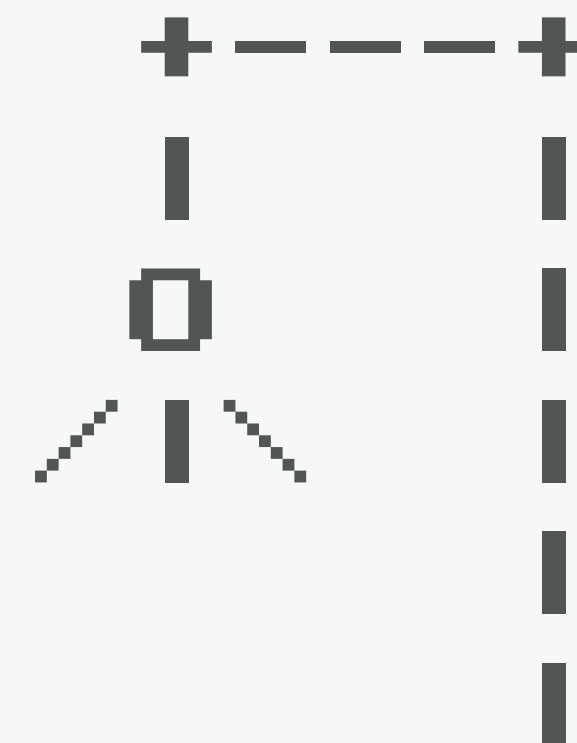
DAVI CELESTINO
LARISSA FERREIRA
RENATO COCA
TENÓRIO VENÂNCIO
OTAVIO JOSHUA



START

ENUNCIADO DO PROJETO

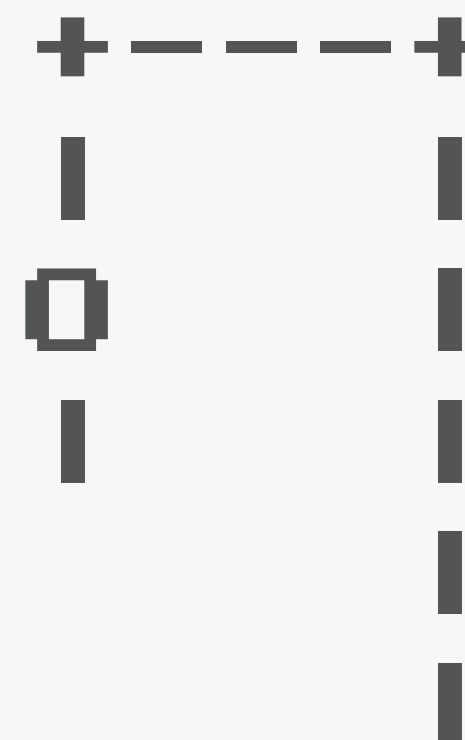
- APLICAÇÃO EM PYTHON
- INTERFACE CLIENTE-SERVIDOR
- SCKETS TCP
 - SOCKET ()
 - BIND ()
 - LISTEN ()
 - ACCEPT ()
 - CONNET ()
 - SENDALL ()
 - RECV ()



DESCRIÇÃO GERAL



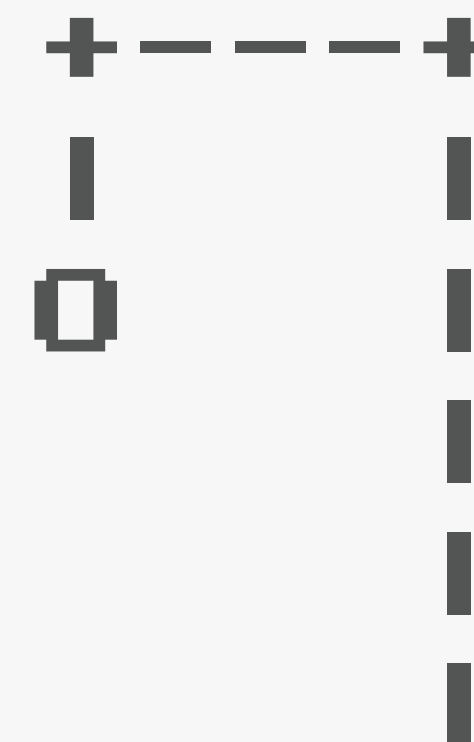
- COMPETITIVO VIA TERMINAL
- JOGADORES
- PALPITES
- PONTUAÇÃO
- TENTATIVAS
- QUANDO TERMINA?





CATEGORIA: PROJETO DE REDES

DEMONSTRAÇÃO



LETRAS ERRADAS: L



ALT TAB

-

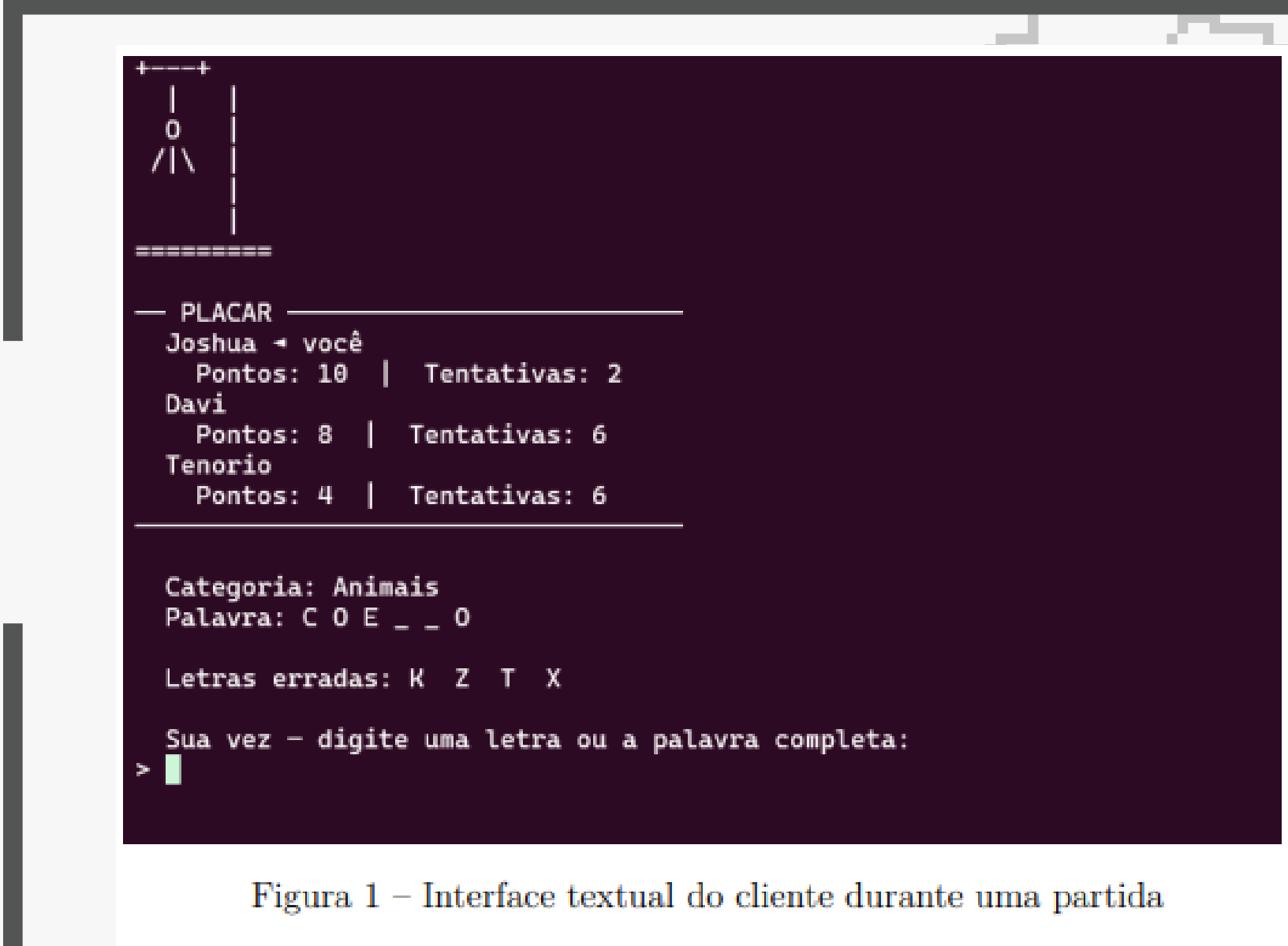
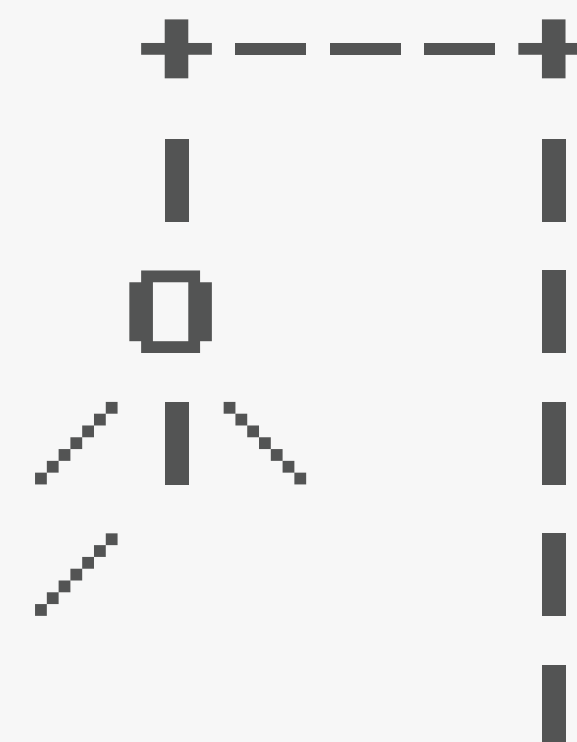


Figura 1 – Interface textual do cliente durante uma partida

USO DE SOCKETS E THREADS



- SERVIDOR: 0.0.0.0
- PORTA 5000
- ENDEREÇO IP
- FUNÇÕES BLOQUEANTES



PROTOCOLO DE COMUNICAÇÃO

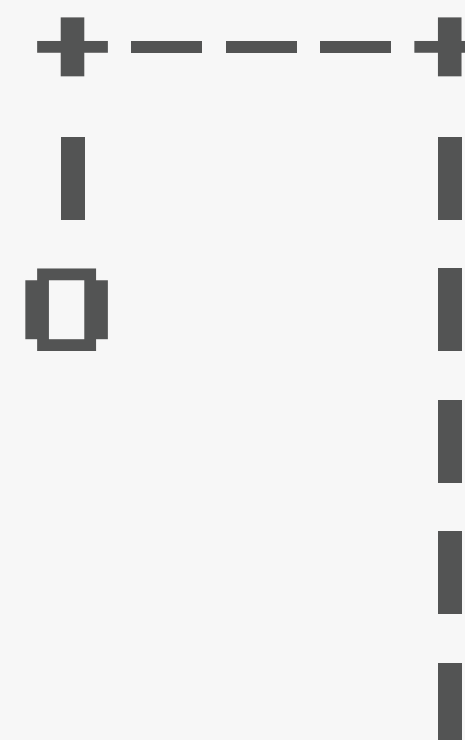


- JSON
- \N
- EXEMPLO JSON:

```
SEND_MSG(SOCK, "GUESS_LETTER", "A")
```

VIRA ALGO COMO:

```
{"TYPE": "GUESS_LETTER", "PAYLOAD":  
"A"}\N
```

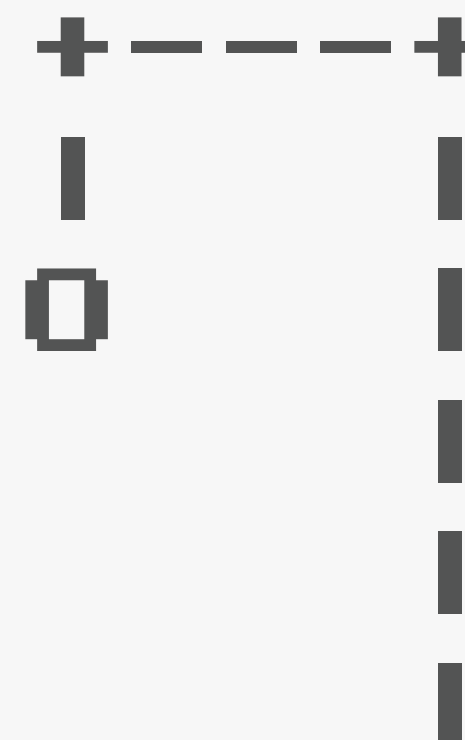


PROTOCOLO DE COMUNICAÇÃO



FLUXO DE MENSAGENS

- | | |
|----------------|-----------------|
| ◦ JOIN | ◦ GUESS_WORD |
| ◦ WELCOME | ◦ CORRECT_GUESS |
| ◦ WAITING | ◦ WRONG_GUESS |
| ◦ GAME_START | ◦ PLAYER_OUT |
| ◦ STATE_UPDATE | ◦ GAME_OVER |
| ◦ GUESS_LETTER | ◦ READY |
| | ◦ ERROR |



PROTOCOL.PY



- MENSAGEM PADRÃO
- SEND_MSG()
- RECV_MSGS()
- CAMADA DA APLICAÇÃO

```
16
17
18
19
20 def send_msg(sock: socket.socket, msg_type: str, data: Any) -> None:
21     """Serialize and send one protocol message."""
22     payload = {"type": msg_type, "payload": data}
23     encoded = (json.dumps(payload, ensure_ascii=False) + "\n").encode("utf-8")
24     sock.sendall(encoded)
25
26
27
28
29
```



PROTOCOL.PY



- MENSAGEM PADRÃO
- SEND_MSG()
- RECV_MSGS()
- CAMADA DA APLICAÇÃO

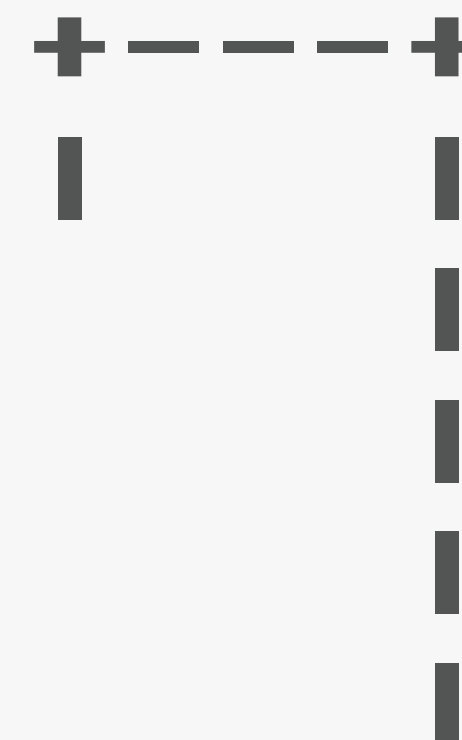
```
34
35 def recv_msgs(sock: socket.socket, buffer: list[str]) -> list[dict[str, Any]]:
36     """Read from socket, accumulate fragments, and return complete messages."""
37     if not buffer:
38         buffer.append("")
39
40     chunk = sock.recv(4096)
41     if not chunk:
42         return []
43
44     buffer[0] += chunk.decode("utf-8")
45     parts = buffer[0].split("\n")
46     buffer[0] = parts.pop()
47
48     messages: list[dict[str, Any]] = []
49     for raw in parts:
50         line = raw.strip()
51         if not line:
52             continue
53         messages.append(json.loads(line))
54
55     return messages
56
```



CLIENT.PY



- PARSE_ARGS()
- SOCKET.CREATE_CONNECTION()
- SEND_MSG(CLIENT_SOCKET, "JOIN", PLAYER_NAME)
- RECV_THREAD()
- RECV_LOOP()
 - RENDER_STATE()
 - RENDER_WAITING()
 - RENDER_GAME_OVER()
- LOOP PRINCIPAL
 - GUESS_LETTER/WORD()
- TRATAMENTO DE ERROS



GAME_SERVER.PY



- ESTADO GLOBAL: GAMESTATE()
- ACCEPT_LOOP()
- HANDLE_CLIENT()
- JOIN
- GUESS_LETTER/WORD()
- READY
- FINALLY DE HANDLE_CLIENT()

```
CONN, ADDR = SERVER_SOCK.ACCEPT()
```

```
CLIENT_THREAD = THREADING.THREAD(  
    TARGET=HANDLE_CLIENT,  
    ARGS=(CONN, ADDR, PLAYER_ID),  
    DAEMON=TRUE  
)  
CLIENT_THREAD.START()
```

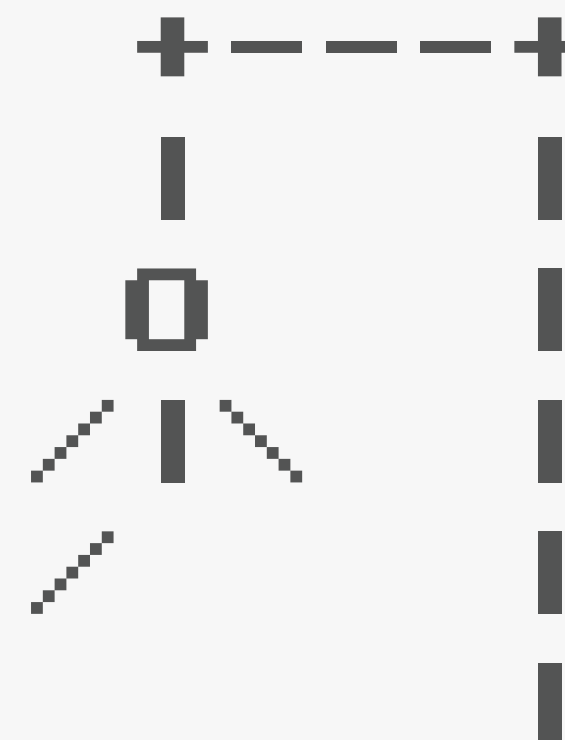
=====



GAME_STATE.PY



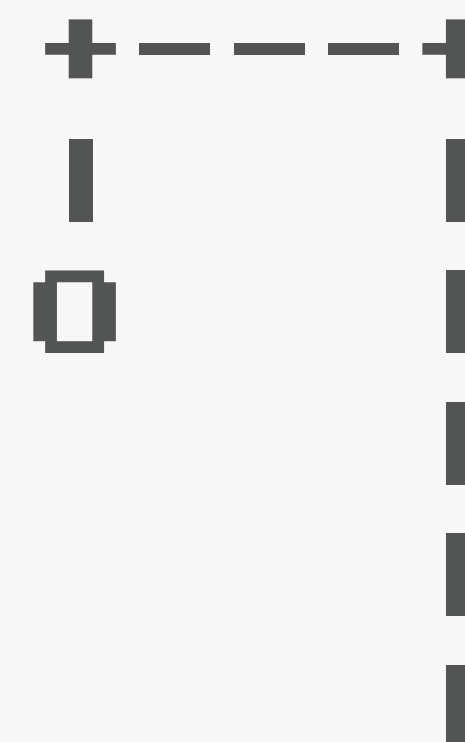
- PLAYER DATA
- THREADING.LOCK()
- ADD/REMOVE_PLAYER()
- START_GAME()
- BROADCAST()
- GET_STATE_PAYLOAD()



DIFICULDADES



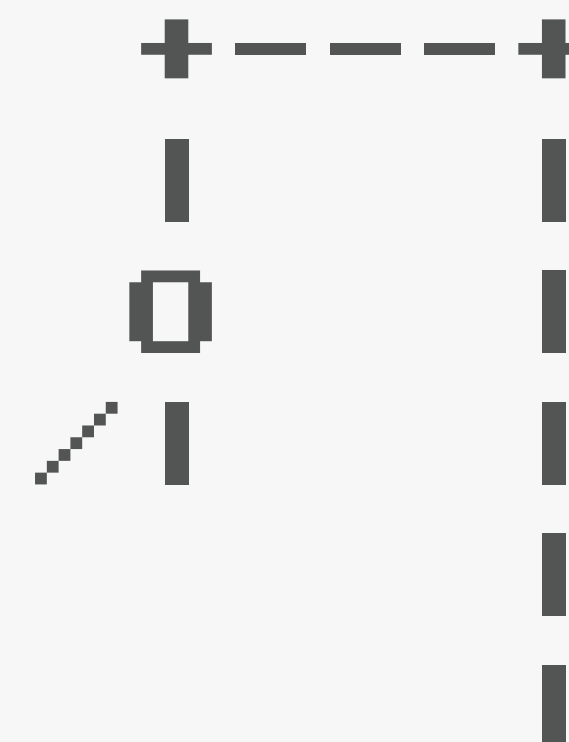
- FLUXO DE BYTES TCP
- CONTROLE DE CONCORRÊNCIA
- TRATAMENTO DE DESCONEXÇÕES
- ESTADO LOCAL DO CLIENTE X ESTADO GLOBAL DO SERVIDOR



MELHORIAS FUTURAS



- AUTENTICAÇÃO DE JOGADORES
- SALAS INDEPENDENTES
- RANKING PERSISTENTE
- RECONECÇÃO DE REDE
- INTERFACE GRÁFICA
- CRIPTOGRAFIA
- LOGS
- CONFIGURAÇÃO DINÂMICA DE JOGADORES/TENTATIVAS

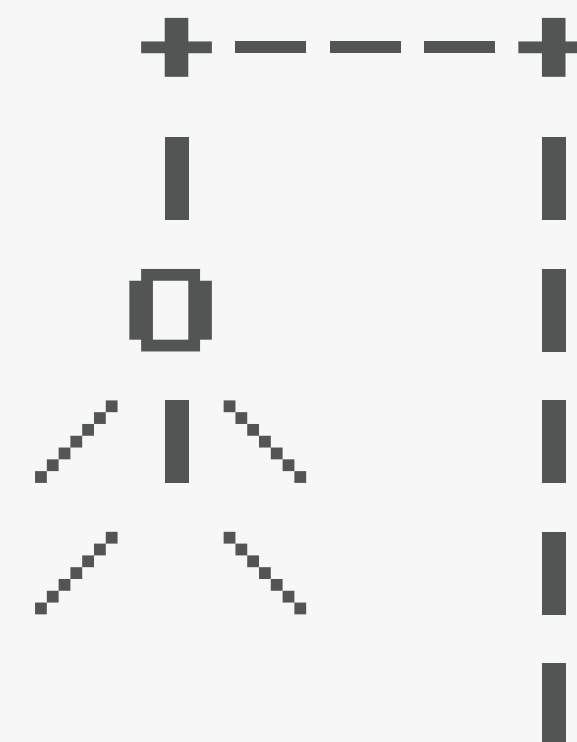


CATEGORIA: PROJETO DE REDES

08_IGADO



LETRAS ERRADAS: L, V, K, U, I



GAME OVER